

A Practical Guide to Mixture of experts

TOPIC -- MIXTURE OF EXPERTS

-- 01 --

Hierarchical MoE Hierarchical mixtures of experts uses multiple levels of gating in a tree. Each gating is a probability distribution over the next level of gateings, and the experts are on the leaf nodes of the tree. They are similar to decision trees. For example, a 2-level hierarchical MoE would have a first order gating function w_i , and second order gating functions $w_{j|i}$ and experts $f_{j|i}$. The total prediction is then $\sum_i w_i(x) \sum_j w_{j|i}(x) f_{j|i}(x)$.

-- 02 --

Deep learning The previous section described MoE as it was used before the era of deep learning. After deep learning, MoE found applications in running the largest models, as a simple way to perform conditional computation: only parts of the model are used, the parts chosen according to what the input is. The earliest paper that applies MoE to deep learning dates back to 2013, which proposed to use a different gating network at each layer in a deep neural network. Specifically, each gating is a linear-ReLU-linear-softmax network, and each expert is a linear-ReLU network. Since the output from the gating is not sparse, all expert outputs are needed, and no conditional computation is performed. The key goal when using MoE in deep learning is to reduce computing cost. Consequently, for each query, only a small subset of the experts should be queried. This makes MoE in deep learning different from classical MoE. In classical MoE, the output for each query is a weighted sum of all experts' outputs. In deep learning MoE, the output for each query can only involve a few experts' outputs. Consequently, the key design choice in MoE becomes routing: given a batch of queries, how to route the queries to the best experts.

-- 03 --

$$\nabla_{\mu_i} \ln f_{\theta}(y|x) = w_i(x) N(y|\mu_i, I) \sum_j w_j(x) N(y|\mu_j, I) (y - \mu_i) \\ \{ \displaystyle \nabla_{\mu_i} \ln f_{\theta}(y|x) = \frac{w_i(x) N(y|\mu_i, I)}{\sum_j w_j(x) N(y|\mu_j, I)} (y - \mu_i) \}$$

-- 04 --

Variants The mixture of experts, being similar to the gaussian mixture model, can also be trained by the expectation-maximization algorithm, just like gaussian mixture models. Specifically, during the expectation step, the "burden" for explaining each data point is assigned over the experts, and during the maximization step, the experts are trained to improve the explanations they got a high burden for, while the gate is trained to improve its burden assignment. This can converge faster than gradient ascent on the log-likelihood. The choice of gating function is often softmax. Other than that, gating may use gaussian distributions and exponential families. Instead of performing a weighted sum of all the experts, in hard MoE, only the highest ranked expert is chosen. That is, $f(x) = f_{\arg \max_i w_i(x)}(x)$. This can accelerate training and inference time. The experts can use more general forms of multivariate gaussian distributions. For example, proposed $f_i(y|x) = N(y|A_i x + b_i, \Sigma_i)$, where A_i, b_i, Σ_i are learnable parameters. In words, each expert learns to do linear regression, with a learnable uncertainty estimate. One can use different experts than gaussian distributions. For example, one can use Laplace distribution, or Student's t-distribution. For binary classification, it also proposed logistic regression experts, with $f_i(y|x) = \begin{cases} \frac{1}{1+e^{\beta_i T x + \beta_i}}, & y=0 \\ \frac{1}{1+e^{\beta_i T x + \beta_i}}, & y=1 \end{cases}$, where $\beta_i, \beta_i, 0$ are learnable parameters. This is later generalized for multi-class classification, with multinomial logistic regression experts. One paper proposed mixture of softmaxes for autoregressive language modelling. Specifically, consider a language model that given a previous text c , predicts the next word x . The network encodes the text into a vector v_c , and predicts the probability distribution of the next word as $\text{Softmax}(v_c W)$ for an embedding matrix W . In mixture of softmaxes, the model outputs multiple vectors $v_{c,1}, \dots, v_{c,n}$, and predict the next word as $\sum_{i=1}^n p_i \text{Softmax}(v_{c,i} W_i)$, where p_i is a probability distribution by a linear-softmax operation on the activations of the hidden neurons within the model. The original paper demonstrated its effectiveness for recurrent neural networks. This was later found to work for Transformers as well.

-- 05 --

Capacity factor Suppose there are n experts in a layer. For a given batch of queries $\{x_1, x_2, \dots, x_T\}$

, each query is routed to one or more experts. For example, if each query is routed to one expert as in Switch Transformers, and if the experts are load-balanced, then each expert should expect on average T / n queries in a batch. In practice, the experts cannot expect perfect load balancing: in some batches, one expert might be underworked, while in other batches, it would be overworked. Since the inputs cannot move through the layer until every expert in the layer has finished the queries it is assigned, load balancing is important. The capacity factor is sometimes used to enforce a hard constraint on load balancing. Each expert is only allowed to process up to $c \cdot T / n$ queries in a batch. The ST-MoE report found $c \in [1.25, 2]$ to work well in practice.